zkDietFormer: Verifiable Transformer Inference with Linear-Cost Attention Proofs via Low-Rank Approximation

First Author[1], Second Author[2], and Third Author[3]

[1] Institute 1
[2] Institute 2
[3] Institute 3

Abstract. The widespread deployment of Transformer-based models via Model-as-a-Service platforms raises a fundamental trust problem: users cannot verify that the provider faithfully executed the claimed model on their input. Zero-knowledge SNARKs (zk-SNARKs) can bridge this gap by letting the provider prove correct inference without revealing proprietary model weights, yet existing approaches face an $O(n^2)$ proving-cost bottleneck rooted in the quadratic self-attention mechanism.

We propose **zkDietFormer**, the first zero-knowledge verification framework designed for low-rank self-attention. By instantiating the Nyström approximation inside the proving circuit, we replace the full $n \times n$ attention computation, including the nonlinear cost induced by the row-wise softmax, with three smaller matrix products involving only $m \ll n$ landmark tokens, reducing the dominant cost to $O(n)$. The prover supplies the pseudoinverse factor as a witness, which the circuit constrains via the Moore–Penrose conditions; Freivalds' single random-vector checks verify all large matrix products, and a commit-carrying SNARK offloads weight commitments outside the circuit while binding every response to a single committed model.

Experiments on five BERT-scale configurations (hidden dimensions 128–768, depths 2–12) show that zkDietFormer extends the maximum provable sequence length by up to 4× over standard attention under the same 128 GB memory budget, with per-layer prove time growing near-linearly in n. Under 26-bit fixed-point arithmetic, the quantized model preserves 98.8% top-1 agreement with its floating-point counterpart.

Keywords: Zero-Knowledge Proofs · Verifiable AI · Low-Rank Attention · Transformer · zk-SNARKs

1 Introduction

Recent breakthroughs in Transformer-based models have revolutionized artificial intelligence, facilitating the widespread deployment of high-performance Large Language Models (LLMs) via Model-as-a-Service (MaaS) platforms. In this setting, model providers typically host large models, while users receive outputs

through APIs or remote inference interfaces. However, this architecture inherently obscures the generation process: users cannot independently verify whether the received output was truly produced from the specific model architecture and weight parameters intended. Users are therefore forced to rely on trust regarding whether the provider substituted a cheaper model, altered outputs before returning them, or silently changed the model during an update. This problem of model execution integrity is particularly critical in high-stakes domains such as finance, healthcare, and security, and becomes increasingly central as demands for transparency and accountability in inference continue to grow.

One may attempt to address this trust issue by requiring the model provider to disclose model parameters for verification. In practice, however, this is infeasible. Model weights are the result of substantial investments in data, compute, and engineering, and are protected as proprietary intellectual property (IP). Consequently, a fundamental dilemma arises: users demand integrity guarantees for the model's outputs, while providers must preserve the privacy of their model parameters to protect their assets.

Zero-knowledge proofs (ZKPs) provide a principled way to reconcile these conflicting requirements. By leveraging ZKPs, a prover (the model provider) can cryptographically prove that an output y is the result of evaluating the model on the user's input x with a secret weight W, i.e., $f(x, W) = y$, without revealing any information about W itself. This enables verifiable inference while maintaining the confidentiality of model parameters.

However, making ZKP-based verifiable AI practically deployable requires efficiently proving computationally intensive deep learning operations, and in modern Transformer architectures the primary bottleneck is self-attention. For an input sequence of length n, self-attention forms an $n \times n$ attention matrix and applies a row-wise softmax over its entirety, incurring $\mathcal{O}(n^2 d)$ linear operations plus an additional $\mathcal{O}(n^2)$ nonlinear cost (Section 3.1).

Probabilistic techniques such as Freivalds' algorithm [11] and Thaler's randomized checks [24] reduce matrix-multiplication verification from $\mathcal{O}(n^2 d)$ to $\mathcal{O}(n^2)$, yet the verification cost remains anchored at $\mathcal{O}(n^2)$. This quadratic term becomes increasingly prohibitive as context windows grow: BERT [9] uses 512 tokens and GPT-3 [2] 2,048, while frontier models now reach 400K (GPT-5 [20]) and 10M (Llama 4 Scout [19]). As n grows, quadratic verification cost renders proof generation for realistic sequence lengths impractical.

Meanwhile, attention matrices often exhibit approximate low-rank structure: with $m \ll n$ landmarks the attention matrix admits a rank-m approximation that reduces inference cost to $\mathcal{O}(n)$ [26, 29]. Yet Verifiable AI has not exploited this structure to reduce proof generation costs, keeping proof complexity at $\mathcal{O}(n^2)$. We bridge this gap by combining low-rank approximation with probabilistic verification to eliminate the quadratic bottleneck in verifying self-attention.

1.1 Our Contribution

In this work, we propose **zkDietFormer**, a verification protocol that eliminates the dominant $\mathcal{O}(n^2)$ bottleneck in proving self-attention by combining a low-

rank representation of the attention matrix, Freivalds' randomized checks, and a commit-carrying SNARK. Our main contributions are summarized as follows:

- **The first verification framework tailored to low-rank self-attention.** To the best of our knowledge, we present the first verification framework specifically designed for low-rank self-attention. We instantiate this framework using the Nyström method, reducing the overall circuit complexity—including both the core matrix operations of self-attention and the nonlinear operations induced by row-wise softmax—to $\mathcal{O}(n)$.
- **Constraint-efficient verification via Freivalds' checks and CC-SNARKs.** We employ Freivalds' algorithm to efficiently verify large matrix computations inside the circuit, while leveraging commit-carrying SNARKs to offload computationally expensive operations outside the circuit, significantly reducing the total number of constraints.
- **Practical implementation and evaluation.** We implement and benchmark **zkDietFormer** across five BERT-family configurations. Per-layer prove time grows near-linearly in n, and our combined optimizations extend the maximum provable sequence length by up to $4\times$ under a fixed 128 GB memory budget.

2 Related Works

Low-Rank Transformer. Standard Transformer attention [25] has $O(n^2)$ complexity. Sparse patterns [6, 1] and memory compression [21] reduce cost but sacrifice full attention. Low-rank methods restore linear complexity while preserving the full attention structure: Linformer [26] projects keys and values into a lower-dimensional space; Performer [7] approximates softmax via random-feature kernelization; and Nyströmformer [29] reconstructs the attention map from m landmark tokens via the Nyström method, achieving higher accuracy than Performer with $O(n)$ complexity.

zk-SNARKs for Matrix Multiplication. Thaler [24] reduces matrix-multiplication verification to $O(n^2)$ via Sumcheck; zkCNN [18] and zkML [5] adopt this for neural-network layers, the latter using Freivalds' algorithm [11]. pvCNN [27] and vCNN [17] cast convolution as a single matrix product verified through Quadratic Matrix Programs. zkLLM [23] achieves prover complexity linear in tensor size via specialized sumcheck protocols, building on SafetyNets [12]. zkMatrix [8] uses Bulletproofs-based acceleration for $O(n^2)$ prover time with $O(\log n)$ verification. However, all these approaches retain $O(n^2)$ prover cost in the matrix dimension [24, 18, 17, 5, 8] or face prohibitively large auxiliary dimensions [23], and nonlinear functions such as softmax require approximation [30].

3 Preliminaries

Notation. We denote the sequence length by n, the hidden dimension by d, and the per-head query/key and value dimensions by d_k and d_v, respectively.

The number of attention heads is h, the number of Transformer layers is L, and the number of landmarks is m, giving a segment length $\ell = n/m$. The circuit precision parameter k_{bits} controls the bit-width of quantized fixed-point arithmetic.

3.1 Self-Attention

Given an input $\mathbf{X} \in \mathbb{R}^{n \times d}$, queries, keys, and values are obtained via learned projections $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, $\mathbf{V} = \mathbf{X}\mathbf{W}^V$ with $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}^V \in \mathbb{R}^{d \times d_v}$. Scaled dot-product attention is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}.$$

In multi-head attention, h heads independently compute attention with per-head dimensions $d_k = d_v = d/h$, and their outputs are concatenated and projected by $\mathbf{W}^O \in \mathbb{R}^{d \times d}$. For clarity, we describe the protocol for a single head with $d_k = d_v = d$; the extension to h heads follows by applying the same construction per head.

3.2 Nystrom-Based Linear Attention

Nyströmformer applies the Nyström method [29]—a technique widely used in matrix approximation—to reduce the computational complexity of the standard Transformer from $O(n^2)$ to $O(n)$. The key idea of the Nyström method is to reconstruct the full matrix approximately by using only a subset of its rows and columns, instead of computing the entire matrix explicitly.

When the softmax matrix S is expressed as a block matrix as shown below,

$$S = \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}) = \begin{bmatrix} A_S & B_S \\ F_S & C_S \end{bmatrix}$$

the Nystrom method approximates the expensive block $C_S \in \mathbb{R}^{(n-m)\times(n-m)}$ by using the smaller blocks $A_S \in \mathbb{R}^{m \times m}, B_S \in \mathbb{R}^{m \times (n-m)}, F_S \in \mathbb{R}^{(n-m) \times m}$ as illustrated below:

$$\hat{S} = \begin{bmatrix} A_S & B_S \\ F_S & F_S A_S^+ B_S \end{bmatrix} = \begin{bmatrix} A_S \\ F_S \end{bmatrix} A_S^+ \begin{bmatrix} A_S & B_S \end{bmatrix}$$

Here, A_S^+ denotes the Moore-Penrose pseudoinverse of A_S.

However, when this approach is applied directly to self-attention, the row-wise normalization property of the softmax function still requires accessing all entries of the $n \times n$ matrix in order to compute the denominators. To address this issue, Nyströmformer partitions the entire sequence into m segments of length $\ell = n/m$ before applying softmax, and computes the mean within each segment to obtain m landmark points for the queries and keys, respectively. These landmarks yield $\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \in \mathbb{R}^{m \times d}$, from which three low-rank factors $\mathbf{Q}\widetilde{\mathbf{K}}^\top \in$

$\mathbb{R}^{n\times m}$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top \in \mathbb{R}^{m\times m}$, and $\widetilde{\mathbf{Q}}\mathbf{K}^\top \in \mathbb{R}^{m\times n}$ are formed. The resulting factors play the same structural role as $\begin{pmatrix} A_S \\ F_S \end{pmatrix}$, A_S^+, and $\begin{pmatrix} A_S & B_S \end{pmatrix}$ in the block decomposition above, but are computed from landmarks rather than extracted from S. Using these matrices, Nyströmformer derives a final approximation that achieves linear computational complexity:

$$\hat{S}V = \text{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right) \text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)^+ \text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right) V$$

The exact computation of the pseudoinverse A_S^+ (e.g., via SVD) is computationally expensive and poorly suited for GPU execution, becoming a practical bottleneck in implementation. To mitigate this issue, Nyströmformer introduces an iterative algorithm that approximates the pseudoinverse using repeated matrix multiplications. The resulting approximation Z^* is then substituted into the final formulation, significantly improving computational efficiency.

$$\hat{S}V = \text{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right) Z^* \text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right) V$$

3.3 zk-SNARKs

A *zero-knowledge succinct non-interactive argument of knowledge* (zk-SNARK) for an NP relation $\mathcal{R}$ consists of three PPT algorithms $\Pi = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$: $(\mathsf{pk}, \mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(1^\lambda, \mathcal{R})$ outputs a proving key and verification key; $\pi \leftarrow \Pi.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; \mathbf{w})$ produces a proof for statement $\mathbf{x}$ and witness $\mathbf{w}$ with $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$; and $0/1 \leftarrow \Pi.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \pi)$ accepts or rejects. A zk-SNARK satisfies *completeness* (an honest prover always convinces the verifier), *knowledge soundness* (an accepting proof implies the prover knows a valid witness), *zero-knowledge* (the proof reveals nothing beyond the statement's truth), and *succinctness* (the proof size is $|\pi| = \mathit{poly}(\lambda) \cdot \mathit{polylog}(|\mathbf{x}| + |\mathbf{w}|)$). Formal definitions appear in [13].

Commit-and-Prove SNARKs (CP-SNARKs). CP-SNARKs [4, 3] let a prover demonstrate that values hidden inside a commitment $\mathsf{cm} = \mathsf{Com}(\mathbf{u}; o)$ satisfy a relation $\mathcal{R}$. The witness is split as $\mathbf{w} := (\omega, \mathbf{u}, o)$, where $\mathbf{u}$ is committed and ω is not. The interface mirrors a plain SNARK except that Setup additionally takes a commitment key ck, Prove takes cm as part of the statement and includes $(\mathbf{u}, o)$ in the witness, and Verify checks both the relation and the commitment consistency.

Commit-Carrying SNARKs (CC-SNARKs). CC-SNARKs [4, 15] extend CP-SNARKs by generating the commitment *during* proving rather than beforehand. The setup outputs a circuit-dependent commitment key ck alongside $(\mathsf{pk}, \mathsf{vk})$. The prover partitions the witness as $\mathbf{w} := (\mathbf{u}, \omega)$ and outputs both a proof π_{cc} and a commitment $\tilde{\mathsf{cm}} = \mathsf{Com.Commit}(\mathsf{ck}, \mathbf{u}; \tilde{o})$. The verifier

checks $\Pi_{\mathsf{cc}}.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \tilde{\mathsf{cm}}, \pi_{\mathsf{cc}})$. This design moves expensive group exponentiations outside the circuit, which is critical for scaling to large Transformer weight matrices.

3.4 Freivalds' Algorithm

Freivalds' algorithm [11, 16] is a simple yet powerful randomized method for verifying a matrix product without recomputing it. Let $\mathbf{X}, \mathbf{Y}, \mathbf{Z} \in \mathbb{F}^{n \times n}$ and consider the claim $\mathbf{X} \cdot \mathbf{Y} = \mathbf{Z}$. Instead of computing $\mathbf{XY}$ in $\mathcal{O}(n^3)$ time, Freivalds samples a uniformly random vector $\mathbf{r} \xleftarrow{\$} \mathbb{F}^n$ and checks

$$\mathbf{X}(\mathbf{Yr}) \stackrel{?}{=} \mathbf{Zr}.$$

This verification runs in $\mathcal{O}(n^2)$ field operations. This algorithm has soundness error: if $\mathbf{XY} = \mathbf{Z}$, it always accepts. Otherwise, when $\mathbf{XY} \neq \mathbf{Z}$, the check passes with probability at most $1/|\mathbb{F}|$, i.e.,

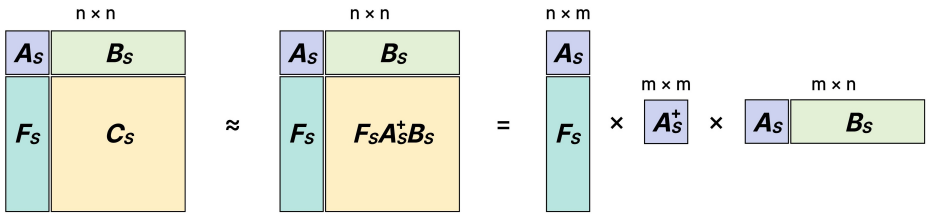
$$\Pr\big[\, \mathbf{X}(\mathbf{Yr}) = \mathbf{Zr} \;\big|\; \mathbf{XY} \neq \mathbf{Z} \,\big] \leq \frac{1}{|\mathbb{F}|}.$$

The same principle extends to rectangular matrices and to the transposed (left-multiplied) variant $\mathbf{r}^\top \mathbf{XY} \stackrel{?}{=} \mathbf{r}^\top \mathbf{Z}$, which we use in Section 5 to verify products of varying dimensions.

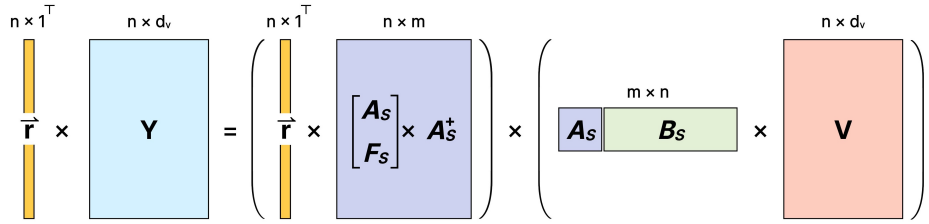
4 Technical Overview

Among the low-rank attention mechanisms reviewed in Section 2, we adopt Nyströmformer [29] because it requires no additional learned projection matrices in the proving circuit and provides numerically stable reconstruction regardless of sequence length. We exclude Performer [7], whose FAVOR+ random feature map introduces nonlinear exponential evaluations that would increase circuit complexity.

As illustrated in Figure 1a, rather than directly handling the $n \times n$ self-attention matrix, we follow the Nyströmformer procedure and decompose it into a sequence of low-dimensional matrices of shapes $n \times m$, $m \times m$, and $m \times n$. We denote the three low-rank factors by $\tilde{F} := \mathrm{softmax}(Q\tilde{K}^\top/\sqrt{d_k}) \in \mathbb{F}^{n \times m}$, $\tilde{A} := \mathrm{softmax}(\tilde{Q}\tilde{K}^\top/\sqrt{d_k}) \in \mathbb{F}^{m \times m}$, and $\tilde{B} := \mathrm{softmax}(\tilde{Q}K^\top/\sqrt{d_k}) \in \mathbb{F}^{m \times n}$, treated formally in Section 5.2. In the block decomposition of Figure 1a, $\tilde{F}$, $\tilde{A}^+$, and $\tilde{B}$ take the roles of $\left[\begin{smallmatrix} A_S \\ F_S \end{smallmatrix}\right]$, A_S^+, and $\left[\begin{smallmatrix} A_S & B_S \end{smallmatrix}\right]$, respectively; they are computed from the landmarks before the softmax rather than extracted from S (cf. Section 3.2), so Figure 1b is obtained from Figure 1a by this substitution. By applying Freivalds' algorithm to this low-rank representation (Figure 1b), we compress the verification workload from $O(n^2)$ to $O(n)$, effectively reducing it to a chain of vector–matrix products of total size $2nm + m^2 + nd_v$: multiplying the random vector by $\tilde{F}$ (nm), then by $\tilde{A}^+$ (m^2), then by $\tilde{B}$ (mn), and finally by $\mathbf{V}$



(a) Nyströmformer matrix approximation



(b) Our **zkDietFormer** Freivalds' Check

Fig. 1: Low-rank matrix multiplication with Freivalds' algorithm

(nd_v). The pseudoinverse factor is instantiated by the iterative approximation Z^* of Section 3.2: the prover computes Z^* outside the circuit and supplies it as a witness, and our protocol verifies in-circuit that Z^* satisfies the Moore–Penrose conditions (Section 5.3) up to quantization tolerance. Since these conditions uniquely characterize the pseudoinverse $\tilde{A}^+$, this check constrains Z^* to a small neighborhood of $\tilde{A}^+$, without ever computing $\tilde{A}^+$ itself.

We embed these low-rank checks in a commit-carrying SNARK (CC-SNARK) architecture (Figure 2): model weights are committed once under a circuit-independent key, and each proof carries a circuit-dependent commitment linked to the global one via CP-linkage, so the verifier needs only constant-time commitment checks (details in Section 5). Because zero-knowledge circuits operate over finite fields, real-valued Transformer computations cannot be represented directly. We therefore quantize the computations in a manner that minimizes precision loss, leveraging the R1CS-friendly quantization technique proposed in [10]. From this point onward, all matrices are understood to be over the finite field $\mathbb{F}$ after quantization.

5 zkDietFormer: Efficient Proofs of Low-Rank Matrix Multiplication via Freivalds' Algorithm

In this section, we present the core protocol of **zkDietFormer**. Our goal is to prove the correctness of Nyström-approximated low-rank self-attention while avoiding any explicit computation of the $n \times n$ attention matrix inside the circuit. Concretely, we design the protocol so that the number of constraints scales

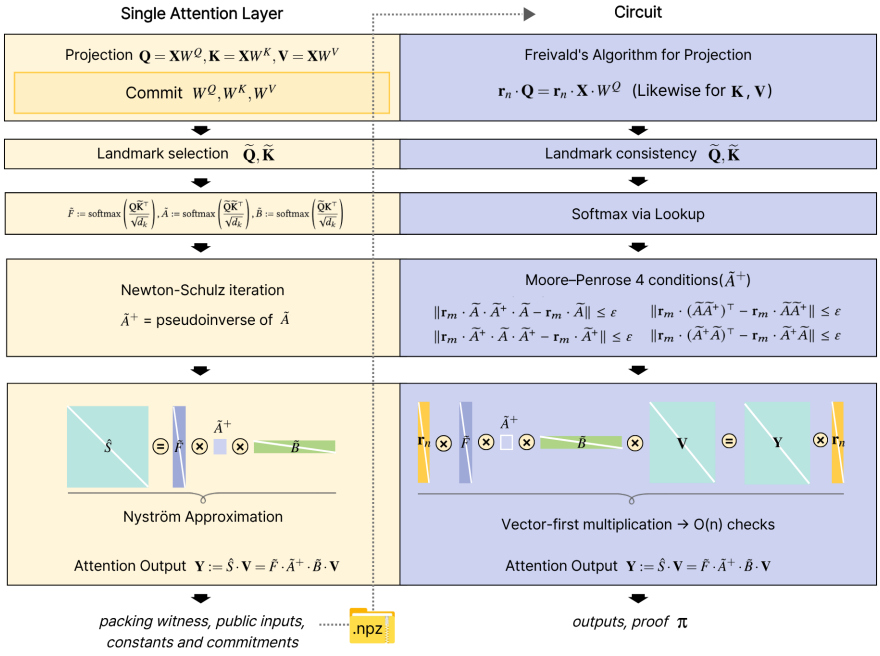


Fig. 2: Overview of Self-Attention and our framework

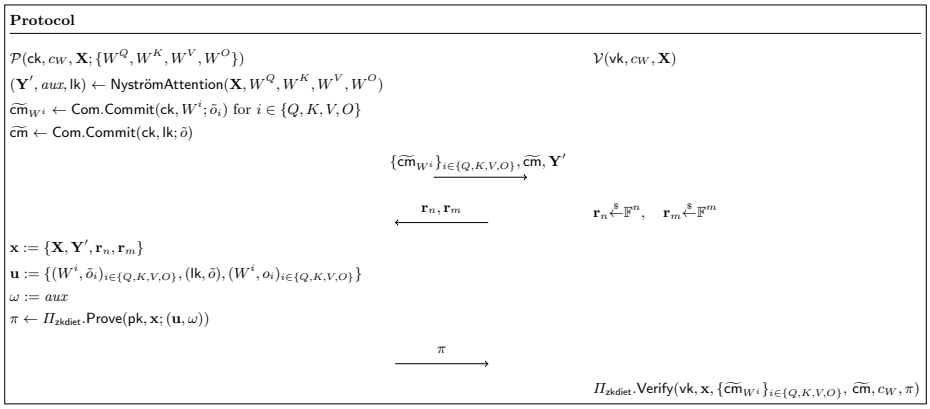


Fig. 3: Our protocol

linearly with the sequence length n. To this end, we adopt a commit-carrying SNARK (CC-SNARK) architecture that pushes computationally expensive operations (e.g., large matrix multiplications and pseudoinverse computation) outside the circuit, and proves only their correctness inside the circuit using a small number of constraints.

5.1 Protocol Overview

Our protocol follows the message flow of Figure 3. Prior to serving any query, the provider publishes a global commitment $c_W \leftarrow \mathsf{Com.Commit}(\mathsf{ck}_{\mathrm{glob}}, W; o)$ to its weights $W = (W^Q, W^K, W^V, W^O)$ under a circuit-independent key $\mathsf{ck}_{\mathrm{glob}}$.

For each query with input $\mathbf{X} \in \mathbb{F}^{n \times d}$, the prover executes $\mathsf{NyströmAttention}$ locally to obtain $\mathbf{Y}'$, non-linear operands $\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$, and auxiliary intermediates *aux*. It then commits to the weights and operands under the circuit-dependent key ck, producing $\widetilde{\mathsf{cm}}_{W^i}$ and $\widetilde{\mathsf{cm}}$, and sends these commitments together with $\mathbf{Y}'$ to the verifier. The verifier replies with random challenges $\mathbf{r}_n \xleftarrow{\$} \mathbb{F}^n$, $\mathbf{r}_m \xleftarrow{\$} \mathbb{F}^m$.

The public instance is $\mathbf{x} := (\mathbf{X}, \mathbf{Y}', \mathbf{r}_n, \mathbf{r}_m)$. The committed witness $\mathbf{u}$ contains the weight matrices and their opening randomness (for both the global and circuit-dependent commitments) plus the non-linear operands; the uncommitted witness $\omega := aux$ contains the remaining intermediates, each constrained by the chain of Freivalds checks linking committed weights to the public output $\mathbf{Y}'$. All commitments are fixed before challenges are sampled, preserving Freivalds soundness.

The prover generates a composite proof π that encapsulates the CC-SNARK proof together with two CP-linkages binding the circuit-dependent commitments $\{\widetilde{\mathsf{cm}}_{W^i}\}$ to the global c_W and $\widetilde{\mathsf{cm}}$ to the lookup arguments. In the non-interactive variant, challenges are derived via the Fiat–Shamir transformation.

Throughout, we write $\tilde{A}^+$ for the pseudoinverse witness, with the understanding that it is the iterative approximation Z^* of Section 3.2, verified via Moore–Penrose conditions (Section 5.3).

5.2 Nyström-Based Low-Rank Self-Attention

We instantiate the Nyström approximation of Section 3.2 inside the proving circuit. Using the landmark matrices $\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \in \mathbb{F}^{m \times d}$ (computed by segment-averaging as described in Section 3.2 and formalized in lines 9–12 of Figure 4), we define the three softmax factors

$$\tilde{F} := \mathrm{softmax}\Big(\tfrac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\Big), \quad \tilde{A} := \mathrm{softmax}\Big(\tfrac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\Big), \quad \tilde{B} := \mathrm{softmax}\Big(\tfrac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\Big).$$

The pseudoinverse $\tilde{A}^+$ is approximated via the iterative procedure of Algorithm 1. The final attention output and output projection are $\mathbf{Y} := \tilde{F} \cdot \tilde{A}^+ \cdot \tilde{B} \cdot \mathbf{V}$ and $\mathbf{Y}' := \mathbf{Y} \cdot \mathbf{W}^O$. Algorithm 2 gives the complete procedure.

5.3 Efficient Verification via Freivalds' Algorithm

Our protocol uses Freivalds' algorithm as a core tool to efficiently verify that the low-rank self-attention computation is performed correctly. In particular, within the zk-SNARK circuit, we employ Freivalds' algorithm to ensure that the following key operations can be verified with linear cost.

Algorithm 1 Compute Moore–Penrose inverse

Input: selected landmark matrix $\tilde{A} \in \mathbb{F}^{m \times m}$
Output: approximate pseudoinverse $\tilde{A}^{+} \in \mathbb{F}^{m \times m}$
$\mathsf{ComputeMPInverse}(\tilde{A})$:

$Z_0 \leftarrow \tilde{A}^{\top} / \left(\|\tilde{A}\|_1 \cdot \|\tilde{A}\|_\infty \right)$
Set T (e.g., $T = 6$ iterations)
for $t \in \{0, \ldots, T-1\}$ **do**
  $Z_{t+1} \leftarrow \frac{1}{4} Z_t \Big(13I - \tilde{A} Z_t \big(15I - \tilde{A} Z_t (7I - \tilde{A} Z_t) \big) \Big)$
endfor
$\tilde{A}^{+} \leftarrow Z_T$
return $\tilde{A}^{+}$

Algorithm 2 Nyström approximation of attention matrix

Input: Input sequence $\mathbf{X} \in \mathbb{F}^{n \times d}$,
Model parameters $W^Q, W^K, W^V, W^O \in \mathbb{F}^{d \times d}$
Output: Attention output $\mathbf{Y}' \in \mathbb{F}^{n \times d}$,
Auxiliary intermediates *aux*,
Non-linear operands $\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$
$\mathsf{NyströmAttention}(\mathbf{X}, W^Q, W^K, W^V, W^O)$:

$\mathbf{Q} := \mathbf{X} W^Q, \quad \mathbf{K} := \mathbf{X} W^K, \quad \mathbf{V} := \mathbf{X} W^V$
$\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \leftarrow \text{SegmentAverage}(\mathbf{Q}, \mathbf{K}, m)$ (cf. Section 3.2)
$\tilde{F} := \text{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d_k}} \right), \quad \tilde{A} := \text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d_k}} \right), \quad \tilde{B} := \text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^{\top}}{\sqrt{d_k}} \right)$
$\tilde{A}^{+} \leftarrow \mathsf{ComputeMPInverse}(\tilde{A})$
$\mathbf{Y} := (\tilde{F} \cdot \tilde{A}^{+})(\tilde{B} \cdot \mathbf{V})$
$\mathbf{Y}' := \mathbf{Y} \cdot W^O$
$aux := (\mathbf{Q}, \mathbf{K}, \mathbf{V}, \widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}}, \mathbf{Q}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \tilde{A}^{+}, \mathbf{Y})$
$\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$
return $(\mathbf{Y}', aux, \mathsf{lk})$

Query/Key/Value matrix generation and output projection. Using the verifier's random vector $\mathbf{r}_n$, we check the relation $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$ by comparing $\mathbf{r}_n^{\top}\mathbf{Q}$ with $\mathbf{r}_n^{\top}(\mathbf{X}\mathbf{W}^Q)$. We verify $\mathbf{K}$ and $\mathbf{V}$ in the same way. Rather than proving the full $n \times d$ matrix product directly, this Freivalds' check reduces verification to computing only the vector-matrix product $\mathbf{r}_n^{\top}\mathbf{X}W^Q$, thereby reducing the complexity from $\mathcal{O}(nd^2)$ to $\mathcal{O}(nd+d^2)$. The output projection $\mathbf{Y}' = \mathbf{Y}\mathbf{W}^O$ is verified

analogously: we check $\mathbf{r}_n^\top \mathbf{Y}' \stackrel{?}{=} (\mathbf{r}_n^\top \mathbf{Y})\, W^O$.

Low-rank matrix operations. We also verify low-dimensional products such as $\mathbf{Q}\widetilde{\mathbf{K}}^\top$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top$, and $\widetilde{\mathbf{Q}}\mathbf{K}^\top$ by multiplying each matrix with a random vector of matching dimension, i.e., $\mathbf{r}_n$ or $\mathbf{r}_m$. For instance, we can certify the correctness of $\mathbf{Q}\widetilde{\mathbf{K}}^\top$ by checking the following equality:

$$\mathbf{r}_n^\top\big(\mathbf{Q}\widetilde{\mathbf{K}}^\top\big) \stackrel{?}{=} (\mathbf{r}_n^\top \mathbf{Q})\, \widetilde{\mathbf{K}}^\top$$

Similarly, the products $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top$ and $\widetilde{\mathbf{Q}}\mathbf{K}^\top$ are validated with linear cost, without explicitly computing the full matrix multiplications inside the circuit.

Moore–Penrose pseudoinverse. Computing $\tilde{A}^+$ inside the circuit is inefficient. Instead, the prover supplies $\tilde{A}^+$ as a witness, and the circuit verifies the four Moore–Penrose conditions (lines 19–22 of Figure 4) up to a bounded residual ε determined by the Newton–Schulz iteration; the concrete tolerance is specified in Section 7.1.

The relations that the prover must satisfy are formalized in Figure 4 as the definition of the circuit relation. In particular, the relation $\mathcal{R}(\mathbf{x}, \mathbf{w})$ takes the public instance $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_m)$ and the witness $\mathbf{w} = (W^Q, W^K, W^V, W^O, \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top, \widetilde{A}^+, \mathbf{Y}, \mathbf{O})$, and enforces all required equalities as well as the softmax computation. All key equalities are expressed in the form of products with the random vectors, accordingly, the circuit computes only vector inner products.

6 Security

In this section, we formalize the security guarantees provided by **zkDietFormer**. We first outline the system and threat models (Section 6.1), then present our security definition and delineate the attack vectors that fall outside our scope (Section 6.2).

6.1 System and Threat Model

Our system involves two parties, a *model provider* and a *user*. The provider holds the trained per-layer weight matrices $(W^i)_{i\in\{Q,K,V,O\}}$ of a Nyström-based Transformer and exposes an inference interface: the user submits an input sequence $\mathbf{X} \in \mathbb{F}^{n\times d}$ and receives the corresponding output $\mathbf{Y}'$. The architectural parameters $\mathsf{zkdiet} = (n, d, h, L, m, k_{\text{bits}})$, including the number of landmarks m and the quantization precision, are public and fixed by the circuit; only the weight values are private. Before serving any query, the provider publishes a single commitment c_W to its weights under a circuit independent key, so that every subsequent response, including responses produced under a circuit instantiated for a different sequence length, is bound to one and the same model.

We denote the resulting functionality by $\mathbf{Y}' \leftarrow \mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}(W, \mathbf{X})$, where $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ is the quantized Nyström attention pipeline of Figure 4, and ε is the residual tolerance admitted for the pseudoinverse factor.

Relation

$\mathcal{R}(\mathbf{x}, \mathbf{w})$:

parse $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_m, \mathbf{Y}'), \mathbf{w} = (\mathbf{u}, \omega)$

parse $\mathbf{u} := \{(W^i, \tilde{o}_i)_{i\in\{Q,K,V,O\}}, (\mathsf{Ik}, \tilde{o}), (W^i, o_i)_{i\in\{Q,K,V,O\}}\}$

parse $\omega = aux$

parse $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n]^\top, \mathbf{K} = [\mathbf{k}_1, \mathbf{k}_2, \ldots, \mathbf{k}_n]^\top$

$\mathbf{r}_n^\top \cdot \mathbf{Q} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot W^Q$

$\mathbf{r}_n^\top \cdot \mathbf{K} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot W^K$

$\mathbf{r}_n^\top \cdot \mathbf{V} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot W^V$

for $j \in \{1, \ldots, m\}$ **do**

$\tilde{\mathbf{q}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{q}_i}{\ell}, \tilde{\mathbf{k}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{k}_i}{\ell}$

endfor

$\widetilde{\mathbf{Q}} \leftarrow [\tilde{\mathbf{q}}_1, \ldots, \tilde{\mathbf{q}}_m]^\top, \widetilde{\mathbf{K}} \leftarrow [\tilde{\mathbf{k}}_1, \ldots, \tilde{\mathbf{k}}_m]^\top$

$\mathbf{r}_n^\top \cdot \mathbf{Q}\widetilde{\mathbf{K}}^\top = \mathbf{r}_n^\top \cdot \mathbf{Q} \cdot \widetilde{\mathbf{K}}^\top$

$\mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top = \mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}} \cdot \widetilde{\mathbf{K}}^\top$

$\mathbf{r}_m^\top \cdot \tilde{\mathbf{Q}}\mathbf{K}^\top = \mathbf{r}_m^\top \cdot \tilde{\mathbf{Q}} \cdot \mathbf{K}^\top$

$\widetilde{F} = \mathrm{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)$

$\widetilde{A} = \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)$

$\widetilde{B} = \mathrm{softmax}\left(\frac{\tilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)$

$\|\mathbf{r}_m^\top \cdot \widetilde{A} \cdot \widetilde{A}^+ \cdot \widetilde{A} - \mathbf{r}_m^\top \cdot \widetilde{A}\| \le \varepsilon$

$\|\mathbf{r}_m^\top \cdot \widetilde{A}^+ \cdot \widetilde{A} \cdot \widetilde{A}^+ - \mathbf{r}_m^\top \cdot \widetilde{A}^+\| \le \varepsilon$

$\|\mathbf{r}_m^\top \cdot (\widetilde{A}\widetilde{A}^+)^\top - \mathbf{r}_m^\top \cdot \widetilde{A}\widetilde{A}^+\| \le \varepsilon$

$\|\mathbf{r}_m^\top \cdot (\widetilde{A}^+\widetilde{A})^\top - \mathbf{r}_m^\top \cdot \widetilde{A}^+\widetilde{A}\| \le \varepsilon$

$\mathbf{r}_n^\top \cdot \mathbf{Y} = \mathbf{r}_n^\top \cdot \widetilde{F} \cdot \widetilde{A}^+ \cdot \widetilde{B} \cdot \mathbf{V}$

$\mathbf{r}_n^\top \cdot \mathbf{Y}' = \mathbf{r}_n^\top \cdot \mathbf{Y} \cdot W^O$

Fig. 4: Relation of **zkDietFormer**

A zero-knowledge verifiable low-rank attention inference (zkDIET) scheme is a tuple of algorithms $\mathsf{zkDIET} = (\mathsf{KeyGen}, \mathsf{Setup}, \mathsf{Commit}, \mathsf{Prove}, \mathsf{Verify})$. We separate key generation into two algorithms because the two commitment layers of our protocol serve different purposes: one binds the model across its entire lifetime, the other binds a single proof.

– $\mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{zkDIET.KeyGen}(1^\lambda, N)$: given the security parameter λ and an upper bound N on the number of field elements to be committed, output a *circuit*

independent commitment key. This key is sampled once and is shared by every circuit configuration the provider may later deploy.

- $(\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{zkDIET.Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}})$: given the relation induced by the architectural parameters zkdiet, output a *circuit dependent* commitment key ck together with a proving key and a verification key. This algorithm is run once per architecture, not per query.
- $c_W \leftarrow \mathsf{zkDIET.Commit}(\mathsf{ck}_{\mathrm{glob}}, W; o)$: commit to the weight matrices $W = (W^i)_{i\in\{Q,K,V,O\}}$ under randomness o. The provider publishes c_W once, before answering any query, and reuses it across all subsequent queries and across all circuit configurations that share these weights.
- $(\mathbf{Y}', \pi) \leftarrow \mathsf{zkDIET.Prove}(\mathsf{pk}, \mathsf{ck}, \mathsf{ck}_{\mathrm{glob}}, \mathbf{X}; W, o)$: evaluate the pipeline on $\mathbf{X}$ outside the circuit to obtain the output $\mathbf{Y}'$, the non-linear operands lk and the auxiliary intermediates aux; invoke the commit-carrying backend, which returns a proof π_{cc} together with the proof-dependent commitments $\widetilde{\mathsf{cm}}_W$ and $\widetilde{\mathsf{cm}}_{\mathsf{lk}}$ generated under ck; and produce a link proof π_{link} asserting that $\widetilde{\mathsf{cm}}_W$ and c_W open to the same weights, and that $\widetilde{\mathsf{cm}}_{\mathsf{lk}}$ and the operand commitment of the lookup argument open to the same lk. The output proof is $\pi := (\pi_{cc}, \widetilde{\mathsf{cm}}_W, \widetilde{\mathsf{cm}}_{\mathsf{lk}}, \pi_{\mathrm{link}})$.
- $\{0,1\} \leftarrow \mathsf{zkDIET.Verify}(\mathsf{vk}, \mathbf{X}, \mathbf{Y}', c_W, \pi)$: accept only if π_{cc} verifies against the public instance, the link proof holds, and the lookup argument accepts.

The verifier's trust anchor is the circuit-independent c_W, not the per-proof commitments; the link proof π_{link} bridges the two, allowing responses across different circuit configurations to be traced to the same model.

Threat Model. The two parties mutually distrust each other. The provider is *malicious*: it may answer with the output of a cheaper or stale model, silently swap weights between queries or between circuit configurations, or fabricate intermediate values such as the pseudoinverse witness in order to make an incorrect output verify. The user is *honest but curious*: it follows the protocol but tries to learn the weights from everything it observes, namely the published commitment, the per-proof commitments, the challenge vectors, and the proof itself. Under this model we target two properties.

- *Inference integrity.* An accepted $\mathbf{Y}'$ is the result of evaluating, under the declared architecture, the very weights fixed by the published commitment c_W, and not an arbitrary value. Since c_W is issued once and is independent of the circuit, this binds every response the provider ever gives to a single model.
- *Model privacy in the proof.* The transcript $\left(c_W, \widetilde{\mathsf{cm}}_W, \widetilde{\mathsf{cm}}_{\mathsf{lk}}, \pi_{cc}, \pi_{\mathrm{link}}\right)$ leaks nothing about W beyond what $\mathbf{Y}'$ itself reveals.

6.2 Security Definition

We keep the notation of Section 6.1 throughout: $\mathcal{R}_{\mathsf{zkdiet}}$ is the relation induced by the architectural parameters $\mathsf{zkdiet} = (n, d, h, L, m, k_{\mathrm{bits}})$, and $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ is the quantized Nyström attention pipeline of Figure 4.

Definition 1 (Zero-knowledge verifiable low-rank attention inference). zkDIET *is secure for* $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ *if it satisfies the following three properties.* Completeness*: an honest prover always convinces the verifier.* Soundness*: no PPT*

adversary can make the verifier accept an output $\mathbf{Y}^* \notin \mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}(W^*, \mathbf{X})$ *for the weights* W^* *bound by its own commitment* $c^*_{W^*}$*, except with negligible probability.* Zero-knowledge*: the proof transcript reveals nothing about the weights* W *beyond what the output* $\mathbf{Y}'$ *itself reveals; formally, there exists a PPT simulator that, given only* $(\mathbf{X}, \mathbf{Y}')$*, produces a transcript computationally indistinguishable from the real one. The formal probability statements and the Real/Ideal experiment are given in Appendix B.*

6.3 Security Analysis

Theorem 1 (informal). *Assume (i) the knowledge soundness and zero-knowledge of the CC-SNARK backend, (ii) the soundness of the lookup argument, (iii) the binding and hiding properties of the commitment scheme, and (iv) the random-oracle model for the Fiat–Shamir hash. Then* zkDIET *satisfies completeness, soundness, and zero-knowledge for* $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ *in the sense of Definition 1. Formal probability statements appear in Appendix B; we sketch the argument below.*

Soundness. Given an accepting transcript, knowledge soundness of the CC-SNARK yields a witness satisfying the relation of Figure 4; the binding of the commitments together with the link proof π_{link} ensures that the weights in this witness coincide with those bound by the published c_W, and the lookup argument guarantees that the non-linear operands $(\tilde{F}, \tilde{A}, \tilde{B})$ are correct evaluations of the quantized softmax. It remains to bound the probability that a witness passes the randomized checks while violating one of the underlying matrix identities. Each equality in lines 6–8, 13–15, and 23–24 of Figure 4 is a rank-one projection of a matrix identity; by Freivalds' bound, a violated identity passes its projected check with probability at most $1/|\mathbb{F}|$. Because the Fiat–Shamir challenges $\mathbf{r}_n, \mathbf{r}_m$ are derived from a hash of all commitments and the public instance, every committed value is fixed before the challenges are determined, so the bound applies to adversarially chosen witnesses. A union bound over the eight projected identities gives a total soundness error of at most $8/|\mathbb{F}| + \mathsf{negl}(\lambda)$, which is negligible over the BLS12-381 scalar field ($|\mathbb{F}| \approx 2^{255}$).

Pseudoinverse tolerance. Lines 19–22 constrain the witness Z^* only up to a residual ε, and only through its projection along $\mathbf{r}_m$. Two observations justify this relaxation. First, each coordinate of a projected residual is a non-constant linear polynomial in $\mathbf{r}_m$; if the underlying entry-wise condition is violated, the projection nevertheless lands inside the tolerance band $[-\varepsilon, \varepsilon]$ with probability at most $(2\varepsilon + 1)/|\mathbb{F}|$, which the union bound above absorbs. Second, at $\varepsilon = 0$ the four Penrose conditions uniquely determine $\tilde{A}^+$, and for $\varepsilon > 0$ standard perturbation bounds give $\|Z^* - \tilde{A}^+\| \leq O(\varepsilon \, \|\tilde{A}^+\|)$, so the attention output of any accepted witness deviates from the exact Nyström output by at most $O(\varepsilon \, \|\tilde{F}\| \, \|\tilde{A}^+\| \, \|\tilde{B}V\|)$. This slack is precisely what the functionality $\mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}$ admits: an accepted output is guaranteed to be an output of the ε-tolerant pipeline, not an arbitrary value. Empirically, the residuals observed in Section 7.1 remain below 4% of the quantization range, so the admitted neighborhood is small in practice.

Zero-knowledge. The simulator $\mathcal{S}_1$ generates the CRS with its trapdoor and publishes commitments to zero, which are indistinguishable from the real ones by the hiding property. Given $(\mathbf{X}, \mathbf{Y}')$, $\mathcal{S}_2$ invokes the zero-knowledge simulators of the CC-SNARK, the link proof, and the lookup argument, and programs the random oracle so that the simulated challenges are consistent with the simulated commitments. Since every remaining transcript element is either public or simulated, the transcript reveals nothing about W beyond $\mathbf{Y}'$.

Out-of-Scope Attacks. While Definition 1 guarantees inference integrity and weight privacy within the proof system, several orthogonal threats fall outside the scope of this work. First, a user issuing many queries may attempt to reconstruct the model weights W from the returned outputs $\mathbf{Y}'$ alone, entirely without relying on the proof. Defenses against such black-box model extraction (e.g., rate limiting, output truncation, or watermarking) are studied independently and can be layered on top of our protocol.

Second, a commitment binds the provider to a single model, but does not guarantee model quality; an adversary could commit to a backdoored or low-accuracy model and continuously produce valid proofs for it. Attesting to the quality or fairness of the committed model requires auxiliary mechanisms (such as proving accuracy on a public labeled dataset), which we leave for future investigation.

7 Experiments

In this section, we evaluate the concrete proof generation costs of **zkDietFormer** and compare them against a standard self-attention baseline. Because our contribution targets the architectural cost of proving attention—i.e., the circuit structure rather than the learned weights—the evaluation focuses on the cost of the circuit itself. We use modified HuggingFace BERT and Nyströmformer configurations (with adjusted hidden dimensions and layer counts) to enable proving across multiple model sizes.

7.1 Implementation

We implement **zkDietFormer** in two parts: a Python-based witness generation pipeline and a Rust-based zero-knowledge proving circuit. Witness data (model weights and intermediate activations) are generated using PyTorch 3.11.15 and HuggingFace Transformers 5.8.0, which instrument the standard `BertForMaskedLM` and `NystromformerForMaskedLM` model classes with forward hooks to capture per-layer activations. The Rust circuit is built on the arkworks library over the BLS12-381 elliptic curve and uses the BLAKE2s hash function for cryptographic operations.

We use LegoGroth16 [4] as our commit-carrying SNARK (CC-SNARK) instantiation. Non-arithmetic operations—softmax exponentials, SigmoidGELU activation, and LayerNorm inverse square root—are evaluated using Lasso [22] lookup arguments.

Pseudoinverse Computation. As described in Section 5.3, the circuit verifies the prover-supplied $\tilde{A}^+$ against the Moore–Penrose conditions up to a bounded residual enforced via element-wise range checks. We empirically evaluated the maximum element-wise residual under a tiny-spec configuration (m=32 landmarks and 12-bit quantization). The observed maximum residuals for the verified conditions range from 1 to 161 relative to the full quantization range $[0, 4095]$. To accommodate these residuals, the circuit enforces bounds of $\epsilon \in \{4, 64, 512\}$, providing a 3× to 5× safety margin above the empirical maximum errors, which strictly remain below 4% of the full range.

Quantization. Zero-knowledge circuits operate over finite fields and cannot represent floating-point arithmetic directly. We therefore quantize all model computations following the R1CS-friendly quantization scheme of ZEN [10]. All weights and activations are converted to 12-bit asymmetric fixed-point representations, and the circuit precision parameter is set to $k_{\text{bits}} = 26$, providing sufficient headroom for accumulated multiplication errors across a full layer. Different operators use different fractional precisions to maintain numerical fidelity: LayerNorm uses 8-bit fractional precision (`FRAC_BITS` = 8); Softmax uses 16-bit fractional precision (`FRAC_BITS` = 16), reflecting the higher dynamic range required by the exponential function; and SigmoidGELU is evaluated via lookup tables of size $2^{16} = 65{,}536$ entries, embedded into the circuit using lookup arguments.

Activation Function Replacement. While our primary focus is on the attention sub-circuit, we implemented and measured the full transformer pipeline (Attention + MLP + LayerNorm); detailed MLP + LayerNorm measurements are reported in Appendix A. One modification required by this end-to-end implementation concerns the GELU activation used in the MLP block. Both BERT and Nyströmformer use GELU [14] as their default activation, typically implemented via the approximation $\text{GELU}(x) \approx 0.5\,x\big(1+\tanh\big[\sqrt{2/\pi}\,(x+0.044715\,x^3)\big]\big)$. We replace GELU in both models with SigmoidGELU, defined as

$$\text{SigmoidGELU}(x) = x \cdot \sigma(1.702\,x),$$

where $\sigma(\cdot)$ denotes the logistic sigmoid. This alternative, originally noted by Hendrycks and Gimpel [14] as a close approximation to GELU, is substantially more circuit-friendly because the sigmoid function can be evaluated via a single lookup table, whereas the tanh-based form requires a more involved computational procedure. The replacement is applied to all layers in both models.

7.2 Experimental Configuration

Hardware. All experiments run on a Mac Studio with an Apple M1 Ultra (20 cores) and 128 GB unified memory (CPU only; no GPU). Timing is CPU wall-clock via Rayon's thread pool; peak memory is the maximum RSS reported by the system `time` utility.

Baselines and Configurations. We compare two HuggingFace [28] architectures—`BertForMaskedLM` [9] ($O(n^2 d)$ attention) and `NystromformerForMaskedLM` [29] ($O(n)$ attention)—across five BERT-family configurations (Table 1). Both share identical (L, d, h) per scale, differing only in the attention kernel.

Model	L	d	h
Tiny	2	128	2
Mini	4	256	4
Small	4	512	8
Medium	8	512	8
Base	12	768	12

Table 1: Model configurations across the standard BERT family (Tiny to Base)

7.3 Results

All reported metrics are *per single layer*: each table entry is the average of the per-layer measurements across all L layers of the model (e.g., L=2 for Tiny). Timing values (setup, commit, prove, verify) and proof size are averaged over the L layers. Memory denotes the peak resident set size (RSS) observed during a single process that proves all L layers sequentially; because it is a process-wide maximum, it is *not* averaged but reported as a single value. We measure six sequence lengths $n \in \{128, 256, 512, 1024, 2048, 4096\}$ with a fixed landmark count m=32 for Nyström attention. This range is chosen to expose the asymptotic advantage of Nyström attention over standard self-attention: since the landmark count m is held fixed while n varies, the Nyström approximation scales as $\mathcal{O}(n)$ in both time and memory [29], whereas standard self-attention scales as $\mathcal{O}(n^2)$. The cost gap between the two thus widens as n grows, and our sweep is designed to capture this transition—from short sequences where the two are comparable, to long sequences where the quadratic baseline becomes prohibitively expensive. For each circuit instance, we report constraint count, setup time, prove time, verify time, peak memory, and two commitment costs: the Groth16 committing time (G16) for committing to the circuit witness, and the Lasso committing time for committing to the lookup operands. The global commitment to the model weights is a one-time cost independent of the number of layers and is therefore excluded from the per-layer metrics; in our measurements it takes approximately 965 ms. Runs that exceed available memory are reported as out-of-memory (OOM); these patterns are themselves informative results, as they indicate the practical scalability boundary of each circuit variant. We evaluate **zkDietFormer** by applying its two constituent techniques—low-rank (Nyström) attention and Freivalds-based matrix-multiplication verification—one at a time, first isolating each technique's individual contribution and then combining both to measure the advantage of the full scheme. In our benchmarks, we pass only cryptographic hashes of the input sequence $\mathbf{X}$ and the attention output $\mathbf{Y}'$ as the public instance, while supplying $\mathbf{X}$ and $\mathbf{Y}'$ themselves as witnesses. This approach avoids the prohibitive setup cost incurred by large public-input vectors. Note that this is a deployment-time configuration; the protocol definition in Figure 3 presents $\mathbf{X}$ and $\mathbf{Y}'$ directly as public instances for clarity.

Low-Rank Attention is Essential for Asymptotic Scaling Fixing the verification method at naive and comparing the BERT-Naive vs. Nys-Naive rows

Table 2: Full 2×2 factorial design—attention kernel (BERT / Nyström) × matrix-multiplication verification (Naive / Freivalds)—on the Tiny configuration (L=2, d=128, h=2). All metrics are per single layer (averaged over L layers). OOM entries (—) indicate that setup exhausted 128 GB; for these runs the Peak RAM column reports the observed process-wide maximum before termination.

				Commit (ms)					
n	Configuration	Constr. (M)	Setup (s)	G16	Lasso	Prove (s)	Verify (ms)	Proof (KB)	RAM (GB)
128	BERT-Naive	15.06	362.83	165.5	159.5	81.21	328.0	900.0	73.04
	BERT-Freivalds	4.97	164.29	167.5	154.5	33.91	302.0	900.0	29.63
	Nys-Naive	13.47	343.81	148.0	265.0	81.20	606.0	1,777.3	67.22
	Nys-Freivalds	4.86	160.51	153.5	255.5	35.73	610.5	1,777.3	29.78
256	BERT-Naive	39.22	911.89	582.0	280.0	337.23	460.5	1,138.9	112.89
	BERT-Freivalds	10.72	407.99	442.0	307.5	85.80	392.0	1,138.9	67.24
	Nys-Naive	26.49	693.76	285.5	314.0	187.00	675.5	1,934.1	112.44
	Nys-Freivalds	9.55	318.63	257.5	296.5	71.92	653.0	1,934.1	54.78
512	BERT-Naive	—	—	—	—	—	—	—	>113 (OOM)
	BERT-Freivalds	24.79	917.80	1,582.5	672.5	216.22	618.0	1,620.5	112.38
	Nys-Naive	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Freivalds	18.93	643.75	468.5	425.0	152.04	781.5	2,202.7	105.35
1024	BERT-Naive	—	—	—	—	—	—	—	>113 (OOM)
	BERT-Freivalds	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Naive	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Freivalds	37.69	1,405.98	1,347.5	592.5	629.29	893.0	2,492.4	112.42

of Table 2 (Tiny, L=2, d=128) isolates the attention kernel's effect. At n=128 the constraint gap is modest (15.06M vs. 13.47M, 1.1×), but by n=256 it widens to 39.22M vs. 26.49M (1.5×) with prove time diverging from 337.23 s to 187.00 s (1.8×). Both exhaust 128 GB at $n \geq 512$. The metrics on which Nyström-naive is worse than BERT-naive—verify time, Lasso commit time, and proof size—reflect the fixed overhead of four Moore–Penrose checks and three additional softmax operations, independent of n.

Freivalds Removes the Constant Factor Holding the kernel at Nyström and comparing Nys-Naive vs. Nys-Freivalds isolates Freivalds' contribution. Across all sequence lengths, Freivalds reduces per-layer constraints by ∼2.8× (e.g., 26.49M→9.55M at n=256), setup time by 2.2×, and prove time by 2.6×. This ratio is stable, confirming a uniform constant-factor improvement. Freivalds extends the feasibility boundary from n=256 to n=1024 (Table 2).

Freivalds Alone Cannot Substitute for Low-Rank Attention Freivalds yields comparable ∼3× constraint reductions on BERT (Table 2), confirming it is a generic optimization. Yet BERT-Freivalds still exhausts 128 GB at $n \geq 1024$ because $\mathcal{O}(n^2)$ complexity remains intact, whereas Nys-Freivalds reaches n=1024. The two optimizations are therefore complementary.

Combined Effect Comparing BERT-Naive with Nys-Freivalds in Table 2 quantifies the joint benefit. At n=256 the combined reduction reaches 4.1× in con-

straints, 2.9× in setup, and 4.7× in prove time, with peak memory dropping from 112.89 GB to 54.78 GB. BERT-Naive is infeasible at $n \geq 512$, while Nys-Freivalds extends to $n=1024$. Full MLP + LayerNorm cost breakdowns are deferred to Appendix A.

Multi-Spec Scaling Figure 5 isolates the cost of proving the attention equation itself—excluding global commitment generation, weight commitment via cc-SNARK, the cp-link proof between the global and weight commitments, and Freivalds-verified **QKV** matrix multiplications—and compares BERT standard attention against Nyström attention across the full BERT model family.

The plot reports per-layer prove time (seconds) against sequence length n on a log-log scale. Each line terminates at the largest n feasible within our 128 GB memory budget. Red lines correspond to BERT and blue lines to Nyström. Within each color family, lighter shades represent smaller configurations and darker shades represent larger ones. Legend abbreviations: Bt/Ny = BERT/Nyström.

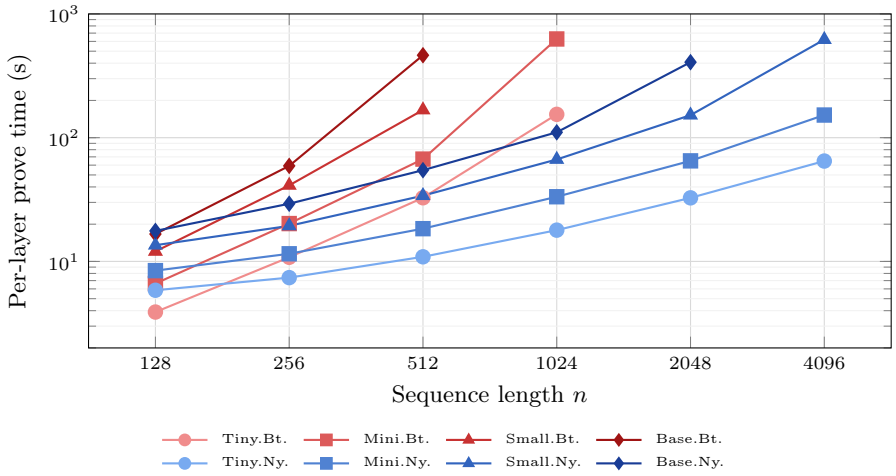


Fig. 5: Per-layer prove time for the attention equation only: BERT (red) vs. Nyström (blue), log-log scale. Medium (L=8, d=512) is omitted as its per-layer cost is identical to Small (d=512).

The most striking feature is the growth-rate gap: BERT's curves (red) rise steeply as n increases, consistent with the $O(n^2)$ attention complexity, while Nyström's curves (blue) grow near-linearly in the attention-only setting, reflecting $O(n)$ complexity. This translates directly into memory exhaustion: at every configuration, BERT exhausts the 128 GB budget at a substantially shorter n than Nyström. The feasibility gap widens as d grows: at Base (d=768, darkest shades), BERT caps at n=512 while Nyström reaches n=2048, a 4× extension.

We omit Medium (L=8, d=512) from the figure because it yields nearly identical per-layer prove times to Small (L=4, d=512)—e.g., 167.03 s vs. 167.48 s for BERT at n=512—confirming depth-invariance: per-layer cost depends only on d and n, not on L. This implies that results extrapolate to deeper architectures by simple multiplication by L.

Quantization Fidelity. Table 3 evaluates the impact of 26-bit fixed-point quantization (k_{bits}=26) on the pretrained uw-madison/nystromformer-512 (vocabulary size 30 000, pretrained on BookCorpus + English Wikipedia) at sequence length n=128 over 10 batches under full cascade evaluation, where quantization errors accumulate through all 12 attention, MLP, and LayerNorm layers. The final quantized output is obtained by applying ZEN's de-quantization to the accumulated fixed-point result, and all fidelity metrics are computed on this de-quantized output. Note that the pretrained model uses m=64 landmarks (its default configuration), whereas the circuit experiments in Section 7.3 use m=32. Top-1 agreement over all positions remains 98.8% and top-5 agreement is 100%, with cosine similarity 0.9975. At the single `[MASK]` position the top-1 agreement drops to 90% but top-5 agreement stays at 100%, indicating that quantization primarily reshuffles near-tie predictions rather than introducing qualitatively wrong answers. These results confirm that 26-bit fixed-point arithmetic provides sufficient precision for faithful ZK-SNARK proving of transformer inference.

Table 3: Quantization fidelity of 26-bit fixed-point arithmetic on a pretrained Nyströmformer (d=768, 12 layers, 12 heads, m=64 landmarks) at n=128.

Top-1 Agree. (%)		Top-5	Cosine	RMSE
All pos.	Mask pos.	All (%)	Sim.	(logit)
98.8	90.0	100.0	0.9975	0.378

7.4 Conclusion and Future Work

Practical impact. Our measurements across all five configurations demonstrate that combining Nyström low-rank attention with Freivalds verification yields multiplicative constraint savings that translate directly into longer provable sequences, while setup memory—not prover time—is the dominant scaling bottleneck (peak RSS ∼118–122 GB on our 128 GB machine).

The MLP frontier. Once attention is optimized, the MLP block emerges as the next memory frontier: at $d \geq 512$, MLP exceeds 128 GB at all sequence lengths (Appendix A). Efficient ZK proving of the MLP block is a natural follow-up direction.

Limitations. *(i)* All measurements use an Apple M1 Ultra without GPU acceleration; GPU or distributed proving would shift relative costs. *(ii)* Our Groth16-based system requires a circuit-specific trusted setup whose memory is our pri-

mary bottleneck; transparent SNARKs would eliminate this at the cost of larger proofs. *(iii)* The MLP sub-circuit remains memory-bound at $d \geq 256$.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Beltagy, I., Peters, M.E., Cohan, A.: Longformer: The long-document transformer (2020), https://arxiv.org/abs/2004.05150
2. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language models are few-shot learners (2020), https://arxiv.org/abs/2005.14165
3. Campanelli, M., Faonio, A., Fiore, D., Querol, A., Rodríguez, H.: Lunar: a toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. Cryptology ePrint Archive, Paper 2020/1069 (2020), https://eprint.iacr.org/2020/1069
4. Campanelli, M., Fiore, D., Querol, A.: LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. Cryptology ePrint Archive, Paper 2019/142 (2019). https://doi.org/10.1145/3319535.3339820, https://eprint.iacr.org/2019/142
5. Chen, B.J., Waiwitlikhit, S., Stoica, I., Kang, D.: Zkml: An optimizing system for ml inference in zero-knowledge proofs. In: Proceedings of the Nineteenth European Conference on Computer Systems. p. 560–574. EuroSys '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3627703.3650088, https://doi.org/10.1145/3627703.3650088
6. Child, R., Gray, S., Radford, A., Sutskever, I.: Generating long sequences with sparse transformers (2019), https://arxiv.org/abs/1904.10509
7. Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., Weller, A.: Rethinking attention with performers (2022), https://arxiv.org/abs/2009.14794
8. Cong, M., Yuen, T.H., Yiu, S.M.: zkmatrix: Batched short proof for committed matrix multiplication. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security. p. 289–305. ASIA CCS '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3634737.3645003, https://doi.org/10.1145/3634737.3645003
9. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding (2019), https://arxiv.org/abs/1810.04805
10. Feng, B., Qin, L., Zhang, Z., Ding, Y., Chu, S.: ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences. Cryptology ePrint Archive, Paper 2021/087 (2021), https://eprint.iacr.org/2021/087

11. Freivalds, R.: Probabilistic machines can use less running time. In: IFIP Congress (1977), https://api.semanticscholar.org/CorpusID:45405368
12. Ghodsi, Z., Gu, T., Garg, S.: Safetynets: Verifiable execution of deep neural networks on an untrusted cloud. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (eds.) Advances in Neural Information Processing Systems. vol. 30. Curran Associates, Inc. (2017), https://proceedings.neurips.cc/paper_files/paper/2017/file/6048ff4e8cb07aa60b6777b6f7384d52-Paper.pdf
13. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) Advances in Cryptology – EUROCRYPT 2016. pp. 305–326. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
14. Hendrycks, D., Gimpel, K.: Gaussian error linear units (GELUs) (2016), https://arxiv.org/abs/1606.08415
15. Jang, B., Jeong, G., Kwon, H., Oh, H., Kim, J.: VECTIS: Efficient batching framework for group-based CP-SNARKs. Cryptology ePrint Archive, Paper 2024/1405 (2024), https://eprint.iacr.org/2024/1405
16. Korec, I., Wiedermann, J.: Deterministic verification of integer matrix multiplication in quadratic time. In: Geffert, V., Preneel, B., Rovan, B., Štuller, J., Tjoa, A.M. (eds.) SOFSEM 2014: Theory and Practice of Computer Science. pp. 375–382. Springer International Publishing, Cham (2014)
17. Lee, S., Ko, H., Kim, J., Oh, H.: vcnn: Verifiable convolutional neural network based on zk-snarks. IEEE Transactions on Dependable and Secure Computing **21**(4), 4254–4270 (2024). https://doi.org/10.1109/TDSC.2023.3348760
18. Liu, T., Xie, X., Zhang, Y.: zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. p. 2968–2985. CCS '21, Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3460120.3485379, https://doi.org/10.1145/3460120.3485379
19. Meta: Llama 4 Model Card. GitHub repository documentation (2024), https://github.com/meta-llama/llama-models/blob/main/models/llama4/MODEL_CARD.md, accessed: 2025-12-31
20. OpenAI: GPT-5 Model. OpenAI API Documentation (2025), https://platform.openai.com/docs/models/gpt-5, accessed: 2025-12-31
21. Rae, J.W., Potapenko, A., Jayakumar, S.M., Lillicrap, T.P.: Compressive transformers for long-range sequence modelling (2019), https://arxiv.org/abs/1911.05507
22. Setty, S., Thaler, J., Wahby, R.: Unlocking the lookup singularity with Lasso. Cryptology ePrint Archive, Paper 2023/1216 (2023), https://eprint.iacr.org/2023/1216
23. Sun, H., Li, J., Zhang, H.: zkllm: Zero knowledge proofs for large language models (2024), https://arxiv.org/abs/2404.16109
24. Thaler, J.: Time-optimal interactive proofs for circuit evaluation. In: Canetti, R., Garay, J.A. (eds.) Advances in Cryptology – CRYPTO 2013. pp. 71–89. Springer Berlin Heidelberg, Berlin, Heidelberg (2013)
25. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need (2023), https://arxiv.org/abs/1706.03762
26. Wang, S., Li, B.Z., Khabsa, M., Fang, H., Ma, H.: Linformer: Self-attention with linear complexity (2020), https://arxiv.org/abs/2006.04768

27. Weng, J., Weng, J., Tang, G., Yang, A., Li, M., Liu, J.N.: pvcnn: Privacy-preserving and verifiable convolutional neural network testing (2023), https://arxiv.org/abs/2201.09186
28. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T.L., Gugger, S., Drame, M., Lhoest, Q., Rush, A.: Transformers: State-of-the-art natural language processing. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. pp. 38–45. Association for Computational Linguistics (2020)
29. Xiong, Y., Zeng, Z., Chakraborty, R., Tan, M., Fung, G., Li, Y., Singh, V.: Nyströmformer: A nyström-based algorithm for approximating self-attention (2021), https://arxiv.org/abs/2102.03902
30. Zhang, Y., Zheng, M., Chen, X., Hu, J., Shi, W., Ju, L., Solihin, Y., Lou, Q.: zkvc: Fast zero-knowledge proof for private and verifiable computing (2025), https://arxiv.org/abs/2504.12217

A MLP + LayerNorm Circuit Costs

A complete transformer layer consists of two sub-circuits: attention and the MLP + LayerNorm block. While the main text focuses on the attention sub-circuit—the target of our zkDietFormer optimizations—we additionally measured the cost of proving the MLP + LayerNorm sub-circuit, whose constraint count is the same for BERT and Nyströmformer because the MLP and LayerNorm dimensions depend only on n and d, not on the attention mechanism. Table 4 reports per-layer average MLP + LayerNorm costs for the Tiny configuration (L=2); all timing and memory figures are the mean of the BERT and Nyström runs. These measurements show that the MLP + LayerNorm sub-circuit and the attention sub-circuit contribute comparable costs at n=1024: per-layer MLP + LayerNorm proving takes 516.62 s versus 629.29 s for Nyström-Freivalds attention. The MLP + LayerNorm block consumes 112.48 GB peak RAM at n=1024, making it a co-bottleneck alongside attention for memory-constrained environments.

For reference, proving a complete Tiny transformer layer (attention + MLP + LayerNorm) at n=1024 requires approximately 2,289.03 s of per-layer setup and 1,145.91 s of per-layer proving (∼57 min total per layer). We report these measurements primarily as a baseline for future work on scalable ZK proving of the full transformer layer.

Table 4: **MLP + LayerNorm** costs for the Tiny configuration (L=2, d=128).

			Commit (ms)					
n	Constr. (M)	Setup (s)	G16	Lasso	Prove (s)	Verify (ms)	Proof (KB)	RAM (GB)
128	0.98	94.23	119.3	147.8	28.38	285.0	755.8	26.66
256	1.96	189.21	217.8	170.3	58.79	311.3	849.9	52.30
512	3.92	385.63	441.3	306.0	137.28	405.5	1,026.2	101.93
1024	7.84	883.05	1,210.3	433.8	516.62	482.5	1,201.7	112.48

B Formal Security Definitions

We state the formal probability conditions for the three security properties of Definition 1. Notation follows Section 6.1.

Completeness. For every $W \in \mathbb{F}^{d\times d}$ and every $\mathbf{X} \in \mathbb{F}^{n\times d}$, for $\mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{KeyGen}(1^\lambda, N)$, $(\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}})$, $c_W \leftarrow \mathsf{Commit}(\mathsf{ck}_{\mathrm{glob}}, W; o)$ and $(\mathbf{Y}', \pi) \leftarrow \mathsf{Prove}(\mathsf{pk}, \mathsf{ck}, \mathsf{ck}_{\mathrm{glob}}, \mathbf{X}; W, o)$,

$$\Pr\big[\mathsf{Verify}(\mathsf{vk}, \mathbf{X}, \mathbf{Y}', c_W, \pi) = 1\big] = 1.$$

Soundness. For any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$ such that

$$\Pr\left[\begin{array}{c} \mathsf{Verify}(\mathsf{vk}, \mathbf{X}, \mathbf{Y}^*, c_W^*, \pi^*) = 1 \\ \wedge\ \ c_W^* = \mathsf{Commit}(\mathsf{ck}_{\mathrm{glob}}, W^*; o^*) \\ \wedge\ \ \mathbf{Y}^* \notin \mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}(W^*, \mathbf{X}) \end{array} \middle| \begin{array}{l} \mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{KeyGen}(1^\lambda, N) \\ (\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}}) \\ (c_W^*, \mathbf{X}, \mathbf{Y}^*, \pi^*) \leftarrow \mathcal{A}(\mathsf{ck}_{\mathrm{glob}}, \mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \\ (W^*, o^*) \leftarrow \mathcal{E}^{\mathcal{A}}(\mathsf{trans}_{\mathcal{A}}) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

Zero-knowledge. Write $\mathsf{Eval}(W, \mathbf{X})$ for the deterministic off-circuit evaluation of Algorithm 2. There exists a PPT simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that for every W and every PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, the two experiments below are computationally indistinguishable (barred symbols denote simulator-produced values):

$\underline{\mathsf{Real}_{\mathcal{A},W}(\lambda)}$

$\mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{KeyGen}(1^\lambda, N)$
$(\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}})$
$o \xleftarrow{\$} \mathcal{O}$; $\ c_W \leftarrow \mathsf{Commit}(\mathsf{ck}_{\mathrm{glob}}, W; o)$
$(\mathbf{X}, st) \leftarrow \mathcal{A}_1(\mathsf{ck}_{\mathrm{glob}}, \mathsf{ck}, \mathsf{pk}, \mathsf{vk}, c_W)$
$\mathbf{Y}' := \mathsf{Eval}(W, \mathbf{X})$
$\pi \leftarrow \mathsf{Prove}(\mathsf{pk}, \mathsf{ck}, \mathsf{ck}_{\mathrm{glob}}, \mathbf{X}; W, o)$
return $\mathcal{A}_2(st, \mathbf{Y}', \pi)$

$\underline{\mathsf{Ideal}_{\mathcal{A},\mathcal{S},W}(\lambda)}$

$(\overline{\mathsf{ck}}_{\mathrm{glob}}, \overline{\mathsf{ck}}, \overline{\mathsf{pk}}, \overline{\mathsf{vk}}, \bar{c}_W, \mathsf{td}) \leftarrow \mathcal{S}_1(1^\lambda, N, \mathcal{R}_{\mathsf{zkdiet}})$
$(\mathbf{X}, st) \leftarrow \mathcal{A}_1(\overline{\mathsf{ck}}_{\mathrm{glob}}, \overline{\mathsf{ck}}, \overline{\mathsf{pk}}, \overline{\mathsf{vk}}, \bar{c}_W)$
$\mathbf{Y}' := \mathsf{Eval}(W, \mathbf{X})$
$\overline{\pi} \leftarrow \mathcal{S}_2(\mathsf{td}, \mathbf{X}, \mathbf{Y}')$
return $\mathcal{A}_2(st, \mathbf{Y}', \overline{\pi})$

$$\Big|\Pr\big[\mathsf{Real}_{\mathcal{A},W}(\lambda) = 1\big] - \Pr\big[\mathsf{Ideal}_{\mathcal{A},\mathcal{S},W}(\lambda) = 1\big]\Big| \leq \mathsf{negl}(\lambda).$$